Channel-Based Programming Q&A Tool

Revised project specification

Next.js UI | Any database | Docker-first grading | Security + uploads

Deadline: April 7, 2026 • 3:00 PM (15:00)

What you're building

A channel-based system for programming questions and threaded discussion

Core user flows

- Browse and create channels
- Post a question/message in a channel
- Reply to posts and replies (threaded)
- Attach screenshots to posts/replies
- Vote (thumbs up/down) on posts and replies
- Search across content and users

Marking priorities

- Clear acceptance criteria (easy to verify)
- Runs from a clean clone via Docker
- Auth enforced for all write actions
- Safe file uploads and input handling
- Reasonable UX: loading/empty/error states
- Evidence of testing (manual or automated)

Tech requirements (flexible, but reproducible)

Choose what fits—document it so it can be graded reliably

Next.js App

UI routes + server rendering (optional)
Client calls your API

App Router or Pages Router

API layer

Route Handlers / API Routes
(or separate Node service)

Auth + validation lives here

Database

SQL or NoSQL
Must include schema/migrations
(or setup script) + seed data

Reproducible in Docker

Documentation requirements (non-negotiable)

- README: run steps, ports, admin credentials, migrations/seed
- .env.example: all required environment variables
- Design report: architecture + DB choice + API endpoints + upload approach
- Test report: 10+ manual cases OR automated tests with evidence

Minimum data model (DB-neutral)

These entities remove ambiguity and make grading consistent

User

```
id
passwordHash
displayName
role
```

Channel

```
id
name (unique)
description?
createdBy
createdAt
```

Post

```
id
channelId
authorId
title
body
createdAt
```

Reply

```
id
postId
parentReplyId?
authorId
body
createdAt
```

Vote

```
id
userId
targetType
targetId
value (+1/-1)
unique(user,target)
```

Attachment

```
id
targetType
targetId
mimeType
sizeBytes
path/url/blobRef
```

Part 1 (30%): Basic system

Channels, posts, replies, screenshots, and a working Docker setup

Acceptance criteria

- Landing page explains the tool
- Channel list + create + select
- Post list + create in a channel
- Reply to a post
- Upload and display screenshots
- Data persists in your chosen DB

Docker-first requirements

- docker-compose starts app + DB
- Schema/migrations applied (auto or documented)
- Seed data available for marking
- No external accounts required
- README includes ports + run steps

Part 2 (20%): Accounts + admin

Auth is required for all write actions; admin can moderate

Required account features

- Sign up, sign in, sign out
- Enforce login to create channels/posts/replies/votes/uploads
- Store passwords as secure hashes
- Show display name on content
- Clear “access denied” UX when logged out

Admin powers

- Remove users
- Remove channels
- Remove posts
- Remove replies

Tip: add confirmation dialogs for destructive actions.

Part 3 (10%): Nested replies + voting

Threaded conversations and consistent rating rules

Nested replies (threading)

- Replies can target posts AND other replies
- Store parentReplyId
- UI shows a threaded tree view
- Within a thread: oldest-first is recommended
- No limit on nesting depth (render recursively)

Voting rules (must enforce)

- One vote per user per target
- States: up (+1), down (-1), neutral
- Users can change and remove votes
- Display counts and/or net score
- Decide: can authors vote on themselves?

Part 4 (10%): Search (measurable)

Implement at least 5 query types + pagination/load-more

Minimum query types (pick ≥ 5)

- Substring search across posts + replies
- Content by a specific author
- User with most posts (with count)
- User with least posts (with count)
- Highest- or lowest-ranked content
- (Optional) filter by channel or tag

Search requirements

- Dedicated search page/section
- Pagination or “load more”
- Document: searched fields + ranking formula
- Add indexes where appropriate
- Results show context + links to content

Security + upload minimums

These requirements protect accounts and keep file uploads safe

Authentication & input safety

- Hash passwords (never plaintext)
- Validate inputs (length limits, required fields)
- Escape/render safely to prevent XSS
- Protect cookie sessions from CSRF (if used)
- Basic rate limiting for login/posting

Screenshot uploads

- Allowed: PNG, JPEG/JPG, WebP
- Max size: 5 MB per image (or document your limit)
- Validate MIME type + extension
- Store safely (DB blob or filesystem)
- Serve via controlled route, not raw execution

UX + accessibility minimums

Small polish items that prevent “it works but feels broken”

UX requirements

- Loading states for data fetches
- Empty states (no channels/posts/replies)
- Clear validation and error messages
- Confirm destructive admin deletes
- Responsive layout (basic mobile support)

Accessibility basics

- Inputs have labels
- Primary flows work with keyboard
- Buttons/links have clear focus states
- Avoid “color only” meaning
- Reasonable contrast

Submission checklist

Make it easy for a marker to run and verify everything

Must include

- docker-compose.yml (+ any Dockerfiles)
- All source files + package.json + lockfile
- DB migrations/schema + seed data
- README: run steps, ports, admin creds, scripts
- .env.example (no secrets)
- 10-minute demo video link

Reports (1 page each)

Design report:

- Architecture + DB choice justification
- API endpoints overview
- Screenshot storage approach
- Key packages + why

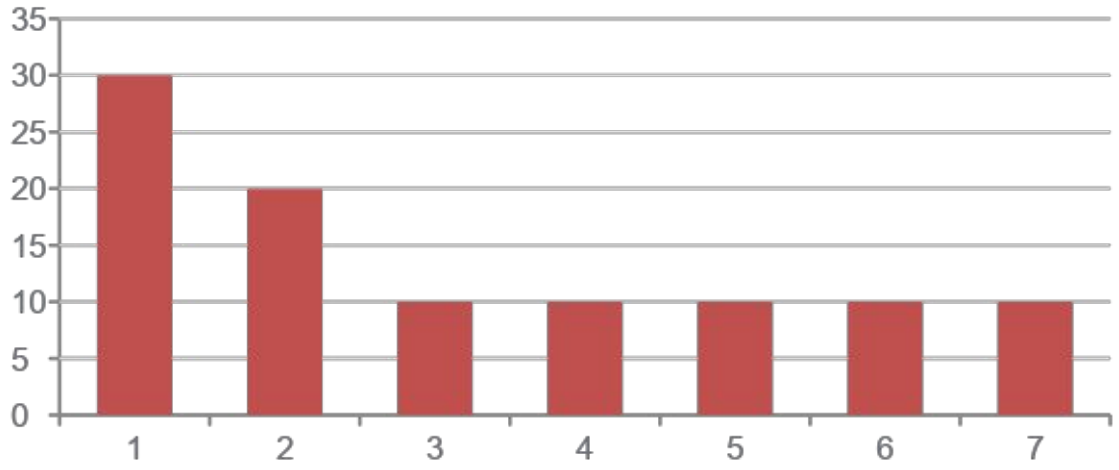
Test report:

- 10+ manual cases OR automated tests + evidence

Scoring + suggested 4-week plan

Hit the rubric early, then add polish and extras

Rubric weights



Suggested timeline

- Week 1: DB schema + API + basic UI (channels/posts)
- Week 2: Auth + admin moderation + uploads
- Week 3: Threaded replies + voting + search
- Week 4: UX polish, testing, reports, demo video

Final deadline

April 7, 2026 • 3:00 PM (15:00)

Aim to have Part 1 done by end of Week 1.